Google Summer of Code

sbi

NUMFOCUS

OPEN CODE = BETTER SCIENCE

Project Proposal for

Neural Network Builder API Refactor

Mentors: Jan Teusen, Nicholas Junge

Large (~350 hours)

By

Satwik Sai Prakash Sahoo

Odisha University of Technology and Research

[redacted] [redacted]

[redacted] [redacted]

Table of Contents

Particulars	Page
1. About Me	02
2. My Open-source Journey with SBI	03 - 05
2.1 Why does SBI resonate with me	03
2.2 My Contributions in SBI	03 – 05
3. Abstract	05
4. Motivation	05
5. Technical Details	06 – 09
5.1. ZScoreConfig Decoupling	06
5.2. Dataclass Validation (Fail-Fast)	07
5.3. Serialization of Complex Fields	08
5.4. Relationship with estimator_configs.py	09
6. Implementation Details	10 - 12
6.1. Resolving Late Validation via __post_init__	10
6.2. Centralizing Z-Score Preprocessing	10
6.3. Backend Routing (nflows vs. Zuko)	10
6.4. Handling Embedding Networks in JSON	11
6.5. Zero-Friction Trainer Integration	11 -12
7. Weekly Timeline For Deliverables	12 - 14

8. Fallback Options and Risk Mitigation	14 - 15
9. Project Outcomes	16
9.1. Value for Users	16
9.2. Value for Maintainers	16
10. Work Commitment	16
11. Post GSoC Plans	17

1. About Me

I am an Electrical Engineering undergraduate at Odisha University of Technology and Research, Bhubaneswar, with a strong foundation in Applied Mathematics and Computer Science. My coursework includes Mathematics I, II, & III, Signals & Systems, Programming in C & Data Structures, Soft Computing, and Object-Oriented Programming.

I have a passion for writing efficient, clean code, backed by a rigorous competitive programming background. On LeetCode, I have achieved the 'Knight' badge with a peak rating of 1994, placing me in the top 5% of users globally. I also actively compete on Codeforces as a 'Pupil' (max rating 1282), where I recently secured a global rank of 2191 in Round 1072 (Div. 3).

I was selected for the Amazon ML Summer School *(August 2025 – September 2026)*, ranking among the top 3,000 candidates out of over 80,000 applicants to receive a month-long mentorship from Amazon scientists about Machine Learning.

I secured 2nd position at a national level hackathon *(October 2025)* organised by IIIT Nagpur for building a HPC enabled MLOps pipeline architecture.

I participated in the IBM QAMP *(Oct 2025 – Jan 2026),* where I collaborated with a global team to develop a Python package that integrates data engineering and ML algorithms with quantum computing which will soon be published on PyPI.

Languages: Python, C, LaTeX
Frameworks: sbi, PyTorch, NumPy, SciPy, Pandas
DevOps & Cloud: Git, Docker, Kubernetes, AWS, GitHub Actions

2. My Open-Source Journey with SBI

2.1 Why does SBI resonate with me?

My interest in SBI stems from the intersection of my academic focus on Electrical Engineering (Signal Processing) and my passion for probabilistic machine learning. Simulation-based inference has helped me for analysing complex physical systems where likelihoods were intractable.

Having worked on HPC and MLOps, I appreciate the library's modular design and its ability to democratize powerful inference methods. I want to contribute to sbi to make these advanced statistical tools more robust, efficient, and accessible to the scientific community.

2.2 My contributions in SBI

SL.	Link	Title	Description
01	#1744 Merged	Use Literal for strict typing of z-scoring options	Replaced loose string arguments with *typing.Literal* in public API to enable static type checking and catch configuration typos at development time.
02	#1711 Merged	Refactor z-scoring tests & enable MNLE support	Refactored tests to fix shape mismatches, resolved variable naming confusion, and enabled testing for Mixed Neural Likelihood Estimation (MNLE) by handling discrete data correctly.
03	#1707 Merged	Fix device mismatch in NPSE marginal computation	Resolved a crash on GPUs by ensuring the internal time tensor used for NPSE marginal statistics is created on the same device as the estimator's registered buffers.
04	#1706 Merged	Fix misleading assertion message in NPE train	Corrected an assertion error message in *NPE.train()* to accurately guide users on whether to resume training or append new simulations during multi-round inference.
05	#1705 Merged	Add max sampling time support to rejection samplers	Implemented a timeout mechanism to prevent infinite loops in rejection sampling and added the *reject_outside_prior* flag to handle highly concentrated posteriors efficiently.
06	#1703 Merged	Add 1D time-series support to Transformer Embedding	Enhanced Transformer Embedding to support 1D time-series inputs automatically and replaced complex configuration dictionaries with user-friendly keyword arguments.

07	#1695 Merged	Docs: How-to guide for time-series embedding	Authored a comprehensive guide demonstrating how to use Causal CNN and Transformer embeddings, providing a clear workflow for users with sequential data.
08	#1753 Merged	Trigger workflow when PR is marked ready for review	Optimized CI/CD pipeline to automatically trigger checks when a draft PR is marked "Ready for Review," improving developer experience.
09	#1750 Merged	Raise informative Runtime Error on empty simulations	Added guard clauses to *train()* methods across all inference families (NPE, NLE, NRE) to catch empty datasets early and provide actionable error messages.
10	#1752 Merged	Re-enable time-dependent z-scoring for Flow Matching	Re-introduced time-dependent normalization for Flow Matching estimators to ensure network inputs remain well-conditioned throughout the diffusion integration process.
11	#1749 Merged	Fix memory bloat in multi-round inference	Resolved a critical memory leak by removing unnecessary deep copies of the neural network during training loops and optimizing the storage of posterior models.
12	#1708 Merged	Default documentation should point to stable	Diagnosed incorrect redirects and guided maintainers to update *Read-The-Docs* configuration to default to stable instead of latest.
13	#1696 Closed	Support scalar time series and simplify config handling in the Transformer Embedding	Proposed enhancing Transformer Embedding to accept 1D time-series inputs directly and replacing the complex dictionary config with explicit keyword arguments for better usability.
14	#1791 Merged	Support shape broadcasting and condition sample_dim in density estimators	Refactored tensor shape handling across all density estimators to support native PyTorch broadcasting, enabling 1-to-N mappings and seamless routing of 3D condition tensors.
15	#1810 Merged	Add IID x support to PosteriorBasedPotential	Added IID support to PosteriorBasedPotential using a vectorized batched forward pass, enabling MCMC and importance sampling for NPE on IID datasets.
16	#1710 Answered	Integrating MLNE with PyMC wrapper	Identified a shape mismatch error where PyMC passed 1D arrays instead of batched tensors; provided a code fix to reshape inputs for MNLE compatibility.

17	#1725 Answered	Very concentrated posterior	Resolved rejection sampling failures due to high posterior concentration by recommending the *reject_outside_prior* flag to bypass 0% acceptance rates.

3. Abstract

The sbi library currently constructs neural density estimators via factory functions (posterior_nn, likelihood_nn, classifier_nn) that accept loose **kwargs, pack them into dictionaries via zip, and return opaque closures. This design has three systemic problems: **(1)** invalid configurations are accepted silently at the factory call and only crash deep inside PyTorch internals during model construction; **(2)** Z-scoring logic is tightly coupled into every builder function, duplicated across nflows and Zuko backends; **(3)** configurations are non-serializable closures, making experiment reproducibility difficult.

This proposal introduces a **three-tier Layered API**:

- **High-level (Quick Experimentation):** posterior_nn("maf") - The existing string-based interface remains unchanged for beginners and rapid prototyping.
- **Mid-level (Production):** DensityEstimatorBuilder(model="nsf", hidden_features=64, ...) with a separate ZScoreConfig(theta="independent", x="structured") - The new recommended standard, offering fail-fast validation and serializable configurations for researchers.
- **Low-level (Full Control):** Direct build_maf(batch_x, batch_y, ...) - The underlying functions remain unchanged for framework developers needing custom module integration.

The Mid-level introduces **fail-fast validation** via Python dataclasses, a **fully decoupled** ZScoreConfig object (with separate theta and x fields, passed to the trainer , not the builder), and a get_config() method for JSON-serializable experiment configs. The builder extends the recently merged _EstimatorConfigBase class (estimator_configs.py), ensuring full backward compatibility through seamless integration with existing trainer checks.

4. Motivation

Designing clean, developer-friendly Python toolkits is something I care deeply about. I am drawn to this refactor because it tackles two classic software engineering frustrations that I have experienced firsthand. The first is the dreaded "late failure." Currently, monolithic factory functions with loose **kwargs allow invalid configurations to pass silently, often resulting in cryptic PyTorch crashes

hours into a training loop. Transitioning to strongly typed dataclasses with fail-fast validation will drastically reduce debugging time and improve the overall user experience.

The second is the entanglement of concerns. sbi currently mixes Z-scoring statistics deeply within the construction of both the nflows and Zuko backends. Untangling this architectural knot is a highly satisfying engineering challenge. Extracting data standardization into a dedicated, immutable ZScoreConfig enforces a strict boundary between preprocessing and the model architecture itself.

Replacing an opaque factory pattern with a modular, Layered API does more than just clean up the internal codebase, it empowers researchers to build reliable, inspectable, and fully reproducible inference pipelines.

5. Technical Details

5.1 ZScoreConfig Decoupling

```
@dataclass(frozen=True)
class ZScoreConfig:
    theta: Literal["independent", "structured",
                   "transform_to_unconstrained", "none"] = "independent"
    x: Literal["independent", "structured",
               "transform_to_unconstrained", "none"] = "independent"

    def __post_init__(self):
        allowed = ("independent", "structured",
                   "transform_to_unconstrained", "none")
        for field_name in ("theta", "x"):
            value = getattr(self, field_name)
            if value not in allowed:
                raise ValueError(
                    f"ZScoreConfig.{field_name} must be one of "
                    f"{allowed}, got '{value}'"
                )
```

The codebase currently defines separate z_score_theta and z_score_x parameters in every factory function (factory.py:87–91, factory.py:241–246). Each independently accepts one of four strategies: "independent", "structured", "transform_to_unconstrained", or "none". The ZScoreConfig dataclass reflects this by providing separate theta and x fields, enabling asymmetric configurations like theta="independent", x="structured", which is critical for time-series data where observation

dimensions are structurally related but parameters are independent.

ZScoreConfig is not a field of the builder so it passed directly to the trainer:

```
trainer = NPE(
    prior,
    density_estimator=DensityEstimatorBuilder(model="nsf", hidden_features=64),
    z_score=ZScoreConfig(theta="independent", x="structured"),
)
```

This follows the principle of separation of concerns: the builder defines the network architecture, while ZScoreConfig defines the data preprocessing strategy. The frozen=True decorator ensures that normalization strategies cannot be accidentally mutated once instantiated, ensuring safety during long training runs.

5.2 Dataclass Validation (Fail-Fast)

```
@dataclass
class DensityEstimatorBuilder(_EstimatorConfigBase):
    model: str = None
    hidden_features: Union[int, Sequence[int]] = 50
    num_transforms: int = 5
    embedding_net: Optional[nn.Module] = None

    def __post_init__(self):
        super().__post_init__()
        valid_models = (
            "maf", "nsf", "mdn", "made", "maf_rqs",
            "zuko_maf", "zuko_nsf", "zuko_naf", "zuko_bpf",
            "zuko_ncsf", "zuko_sospf", "zuko_unaf", "zuko_gf",
            "zuko_nice", "mnle", "mnpe",
        )
        if self.model not in valid_models:
            raise ValueError(
                f"model must be one of {valid_models}, got '{self.model}'"
            )
        if not isinstance(self.hidden_features, (int, list, tuple)):
            raise TypeError(
                f"hidden_features must be int or sequence, "
```

```
            f"got {type(self.hidden_features).__name__}"
        )

    def __call__(self, theta: Tensor, x: Tensor) -> nn.Module:
        return self.build(theta, x)

    def build(self, theta: Tensor, x: Tensor) -> nn.Module:
        pass
```

Key design decisions:

- **No z_score fields**: Z-scoring is a separate concern owned by ZScoreConfig, not the network architecture builder.
- **Extends _EstimatorConfigBase**: Inherits the from_kwargs(), to_dict(), and extra_kwargs infrastructure from the estimator_configs.py. This avoids duplication and ensures consistency with the existing config hierarchy.
- **Expanded valid_models**: The list includes all 15 entries from the model_builders registry (factory.py:46–63).
- **__call__ alias**: Natively fulfills the Callable requirement enforced by check_estimator_arg (user_input_checks.py:712–719), which asserts isinstance(estimator, Callable) and not isinstance(estimator, nn.Module). This allows the builder to slot directly into existing inference pipelines.

5.3 Serialization of Complex Fields

```
def get_config(self) -> dict:
    raw = asdict(self)
    if isinstance(raw.get("hidden_features"), tuple):
        raw["hidden_features"] = list(raw["hidden_features"])
    if isinstance(raw.get("embedding_net"), nn.Module):
        net = raw["embedding_net"]
        raw["embedding_net"] = {
            "type": type(net).__name__,
            "module": type(net).__module__,
            "repr": repr(net)
        }
    return raw
```

Because custom nn.Module objects (like embedding networks) are not natively JSON-serializable, the get_config() method safely detects PyTorch modules and extracts their metadata and representation, ensuring the entire configuration dictionary can be serialized.

This method leverages dataclasses.asdict() to recursively unpack standard fields, while safely transforming complex PyTorch objects. This capability is important for experiment reproducibility. It allows researchers to easily export the exact network architecture configuration to a JSON file alongside their trained model weights. When an embedding_net is present, capturing its class type, module origin, and repr() provides a complete, inspectable fingerprint of the custom feature extractor used during the experiment.

5.4 Relationship with estimator_configs.py

Recently _EstimatorConfigBase, ConditionalFlowConfig, and ClassifierConfig were merged into sbi/neural_nets/net_builders/estimator_configs.py. These configs replace the error-prone dict(zip(...), **kwargs) pattern in the factory functions. The DensityEstimatorBuilder extends this infrastructure rather than replacing it.

The class hierarchy:

```
_EstimatorConfigBase          ← provides from_kwargs(), to_dict(), extra_kwargs
├── ConditionalFlowConfig     ← internal validation layer for flow factory kwargs
├── ClassifierConfig          ← internal validation layer for classifier factory kwargs
├── MarginalFlowConfig        ← internal validation layer for marginal factory kwargs
├── DensityEstimatorBuilder   ← NEW: public-facing builder with build(), get_config(), __call__
└── _VectorFieldBaseConfig    ← base for ScoreEstimatorConfig, FlowEstimatorConfig
```

ConditionalFlowConfig serves as an **internal validation layer** used by the factory functions (posterior_nn, likelihood_nn) to validate kwargs before passing them to low-level builders (verified in factory.py:301, factory.py:225). The DensityEstimatorBuilder sits **one layer above** as the **public API** that users interact with directly. Internally the builder's build() method delegates to the appropriate low-level builder via the model_builders registry, passing its validated fields as explicit keyword arguments.

This layered design preserves the recent estimator_configs.py work while adding user-facing improvements: fail-fast __post_init__ validation, JSON-serializable get_config(), and __call__-based trainer integration.

6. Implementation Details

6.1 Resolving Late Validation via __post_init__

Under the current architecture, invalid configurations fail late. Passing an invalid argument like hidden_features='invalid' to the current factory functions returns a closure with no validation error. It only fails at build time deep in the PyTorch execution stack with a generic exception.

In the proposed state, invalid configurations are caught immediately at instantiation using Python dataclasses. By leveraging the __post_init__ magic method, we enforce strict runtime type and value checking before the builder is even fully instantiated. Instantiating a builder with invalid types raises a TypeError, and unsupported model strings raise a ValueError, preventing wasted compute time and cryptic tracebacks.

6.2 Centralizing Z-Score Preprocessing

Currently, Z-scoring logic is duplicated across backends. For nflows, each builder (build_maf, build_nsf, etc.) calls z_score_parser and standardizing_transform internally (flow.py:77–80). For Zuko, build_zuko_flow (flow.py:1046) calls _prepare_x_transforms (flow.py:1190) which calls z_score_parser and standardizing_transform_zuko , achieving the exact same thing through a different code path.

Since z-scoring is now a **separate concern** owned by ZScoreConfig and passed to the trainer (not the builder), the implementation will centralize this logic. The trainer's train() flow will:

- Evaluate the ZScoreConfig to determine z-scoring strategies for theta and x.
- Construct the appropriate standardizing transforms using the shared z_score_parser and z_standardization utilities (sbiutils.py:155–200, sbiutils.py:286–325).
- Pass these pre-configured transforms to the builder's build() method, which delegates to the backend builders.

This eliminates the duplicated z-score parsing across 14+ builder functions in flow.py and enforces a strict separation of concerns between data standardization and neural network composition.

6.3 Backend Routing (nflows vs. Zuko)

The build(self, theta, x) method acts as a smart router to the underlying legacy builders via the model_builders registry (factory.py:46–63) to preserve numerical parity.

```
def build(self, theta: Tensor, x: Tensor, **z_score_kwargs) -> nn.Module:
    if self.model not in model_builders:
```

```
        raise NotImplementedError(f"Model {self.model} is not implemented")
    return model_builders[self.model](
        batch_x=theta,
        batch_y=x,
        hidden_features=self.hidden_features,
        num_transforms=self.num_transforms,
        embedding_net=self.embedding_net,
        **z_score_kwargs,
    )
```

By explicitly mapping the dataclass fields to the expected backend arguments, we remove the reliance on opaque **kwargs unpacking while maintaining support for both nflows and Zuko libraries. The z_score_kwargs are injected by the trainer after evaluating the ZScoreConfig, not stored on the builder itself.

6.4 Handling Embedding Networks in JSON

dataclasses.asdict() recursively converts nested dataclasses into dictionaries, but it fails on PyTorch nn.Module instances because they are not JSON-serializable.

To solve this, the get_config() method intercepts any field containing an nn.Module. It converts the module into a metadata dictionary containing the class name, module path, and string representation.

Calling get_config() on a configured builder with a custom PyTorch module yields a standard, fully serializable dictionary ready for experiment tracking:

```
{
  "model": "maf",
  "hidden_features": 64,
  "num_transforms": 5,
  "embedding_net": {
    "type": "MyEmbedding",
    "module": "__main__",
    "repr": "MyEmbedding(\n  (net): Sequential(...))"
  }
}
```

6.5 Zero-Friction Trainer Integration

The dispatch pattern for sbi trainers is executed in the __init__ block of the five main base classes (NPE, NLE, NRE, VFPE, Marginal). They all utilize the check_estimator_arg function located in

utils/user_input_checks.py to validate the user's input.

Because check_estimator_arg enforces isinstance(estimator, Callable) and not isinstance(estimator, nn.Module), the DensityEstimatorBuilder passes this check via its __call__ implementation, which aliases directly to build().

```
check_estimator_arg(density_estimator)
if isinstance(density_estimator, str):
    self._build_neural_net = posterior_nn(
        model=density_estimator,
        z_score_theta=z_score.theta,
        z_score_x=z_score.x,
    )
else:
    self._build_neural_net = density_estimator
    self._z_score_config = z_score
```

This guarantees backward compatibility. The base classes seamlessly invoke density_estimator(theta, x), which natively triggers the .build(theta, x) routine. The z-score configuration is threaded separately from the network architecture, matching the decoupled design.

7. Weekly Timeline For Deliverables

Community Bonding Period (May 1 - May 24)

Milestone: Familiarization and finalization of API design.

- Deep-dive into zuko and pyknos internals to map all builder function signatures.
- Finalize the ZScoreConfig and DensityEstimatorBuilder API design with mentors.
- Draft the deprecation strategy for factory-function arguments.

Week 1 (May 25 - May 29)

Milestone: Implementation of core ZScoreConfig and BaseBuilder abstractions.

- Finalize ZScoreConfig dataclass with full validation.
- Define the abstract BaseBuilder with __post_init__ validation, build(), and get_config().
- Write unit tests for config validation.

Week 2 (June 01 - June 05)

Milestone: Concrete builders for nflows-based models.

- Implement concrete builders for maf, maf_rqs, nsf, made, and mdn.
- Ensure build() produces identical networks to current factory functions via numerical comparison tests.

Week 3 (June 08 - June 12)

Milestone: Concrete builders for Zuko-based models.

- Implement builders for zuko_maf, zuko_nsf, zuko_naf, zuko_ncsf, zuko_sospf, zuko_unaf, zuko_gf, and zuko_bpf.
- Verify Zuko argument mapping is correctly aligned.

Week 4 (June 15 - June 19)

Milestone: Score, flow-matching, and classifier builders.

- Implement ScoreEstimatorBuilder and FlowMatchingEstimatorBuilder.
- Implement ClassifierBuilder for ratio estimation networks.
- Ensure EmbeddingNet modules are correctly processed.

Week 5 (June 22 - June 26)

Milestone: Serialization and reconstruction features.

- Implement get_config() module-handling for all builders.
- Implement from_config(dict) class method for reconstruction from saved configs.
- Add JSON/YAML round-trip tests.

Week 6 (June 29 - July 03)

Milestone: Integration verification within inference classes.

- Verify integration within NPE, NLE, NRE, VFPE, and Marginal trainers.
- Ensure the __call__ implementation passes check_estimator_arg flawlessly.
- Run backward compatibility tests for all legacy inference execution paths.

Midterm evaluation (July 6 - 10)

Deliverables:

- All DensityEstimatorBuilder classes implemented and tested.
- Custom module serialization functional.
- Existing test suite passing with zero regressions in trainer integration.

Week 8 (July 13 - July 17)

Milestone: Refactoring internal factory functions.

- Rewrite posterior_nn, likelihood_nn, classifier_nn to instantiate builders internally & call .build().

- Add FutureWarning for direct **kwargs usage.

Week 9 (July 20 - July 24)

Milestone: Decoupling Z-scoring from builder pipelines.

- Extract Z-scoring from builder flows into a shared preprocessing layer driven by ZScoreConfig.
- Remove duplicated z_score_parser and standardizing_transform calls from individual builders.

Week 10 (July 27 - July 31)

Milestone: Advanced preprocessing constraints.

- Extend ZScoreConfig with optional distribution field for constrained-to-unconstrained transforms.
- Integrate with existing _prepare_x_transforms logic.

Week 11 (Aug 03 - Aug 07)

Milestone: Public API documentation and migration guides.

- Write comprehensive docstrings for all public classes and methods.
- Create a migration guide detailing the transition from factory functions to builders.
- Add a tutorial notebook demonstrating the three-tier API.

Week 12 (Aug 10 - Aug 14)

Milestone: Advanced documentation and final code polish.

- Document get_config() / from_config() workflow for experiment reproducibility.
- Execute final code cleanup and linting pass.

Final Submission (Aug 17 - Aug 24)

Deliverables:

- Final round of integration testing across all inference methods completed.
- All remaining reviewer feedback addressed.
- Finalize PR with complete test coverage, API documentation, and migration guide.

8. Fallback Options and Risk Mitigation

Risk 1: Z-score decoupling breaks backend-specific edge cases.

The transform_to_unconstrained z-scoring option requires an x_dist parameter (a PyTorch

Distribution with a .support attribute) that is only relevant for Zuko flows (flow.py:1210–1222). If decoupling z-scoring from the builders exposes edge cases where the transform requires backend-specific context:

- **Mitigation**: Keep z-scoring logic in the low-level builders as a fallback. The ZScoreConfig would pass z_score_x and z_score_y strings through to the existing builder functions (which already handle them correctly), rather than pre-computing transforms. This preserves numerical parity while still achieving the API-level separation of concerns.
- **Escalation**: If backend-specific z-score transforms prove inseparable from the builder, I will document this as an architectural constraint and limit the refactor to the three strategies that are backend-agnostic ("independent", "structured", "none"), deferring "transform_to_unconstrained" to a follow-up PR.

Risk 2: Backward compatibility issues require redesign.

check_estimator_arg (user_input_checks.py:712–719) currently asserts the estimator is either a str or Callable (but not nn.Module). The DensityEstimatorBuilder's __call__ method satisfies this gate. However, if downstream code uses isinstance checks or functools.partial on the estimator (e.g., npe_a.py:229–230 wraps _build_neural_net with partial), the builder may need additional protocol support.

- **Mitigation**: Design the builder to be transparent to functools.partial by ensuring build() accepts **kwargs for additional arguments. Write a dedicated backward compatibility test suite that exercises all five trainer families (NPE, NLE, NRE, VFPE, Marginal) with both legacy closures and the new builder, including the SNPE-A partial path.
- **Escalation**: If the builder cannot be made fully transparent, add a to_callable() method that returns a plain closure wrapping build(), providing a drop-in replacement for the legacy factory output.

Risk 3: Scope creep from the estimator_configs.py integration.

The recently merged config classes (ConditionalFlowConfig, ClassifierConfig, etc.) have their own field sets and validation logic. If aligning the builder's fields with these configs creates unexpected conflicts:

- **Mitigation**: The builder delegates to model_builders[model](...) with explicit kwargs, bypassing ConditionalFlowConfig entirely. The two systems coexist independently , the builder for the public API, the configs for internal factory validation.
- **Buffer weeks**: Weeks 5–6 (serialization and integration verification) serve as explicit buffer time. If earlier phases overrun, serialization features (from_config()) can be deferred to a follow-up PR without blocking the core deliverables.

9. Project Outcomes

9.1 Value for Users

- **Fail-fast validation**: Invalid configurations (wrong model names, incorrect types for hidden_features) are caught at construction time with clear error messages, not hours into a training loop with cryptic PyTorch stack traces.
- **Inspectable configurations**: Researchers can call builder.get_config() to export the exact network architecture as JSON for experiment tracking, paper appendices, and reproducibility , something impossible with the current opaque closures.
- **Asymmetric z-scoring**: The new ZScoreConfig(theta="independent", x="structured") API allows researchers working with time-series or image data to apply different standardization strategies per dimension, with a clean, documented interface.
- **Gradual adoption**: The three-tier API ensures beginners can still write NPE(prior, "maf") while advanced users get full control via DensityEstimatorBuilder.

9.2 Value for Maintainers

- **Reduced code duplication**: Z-score parsing logic is currently duplicated across 14+ builder functions in flow.py (once for nflows via z_score_parser/standardizing_transform, and again for Zuko via _prepare_x_transforms/standardizing_transform_zuko). Centralizing this in ZScoreConfig eliminates this duplication.
- **Unified config hierarchy**: The builder extends _EstimatorConfigBase, keeping the class hierarchy consistent with the recently merged estimator_configs.py infrastructure.
- **Easier onboarding**: New contributors can understand the builder API via typed dataclass fields rather than tracing through dict(zip(...)) patterns and opaque closures.
- **Deprecation path**: FutureWarning on legacy **kwargs usage provides a clear migration timeline without breaking existing code

10. Work Commitment

I am fully committed to the Google Summer of Code timeline. As a university student, my summer break aligns perfectly with the GSoC coding period. I have no conflicting internships, jobs, or academic coursework during this time.

I plan to dedicate 30+ hours per week to the project. I have a reliable development setup and high-speed internet to ensure consistent communication and productivity. My top priority is to produce high-quality work, iterate on feedback, and hit all deadlines effectively.

I'm highly motivated and excited to contribute to this project, and I am confident that my availability and dedication will allow me to meet expectations effectively.

11. Post-GSoC Plans

A significant refactor or feature addition requires long-term vigilance. My journey with SBI will not end with GSoC. I intend to remain a long-term contributor to the library.

SBI is a critical tool in my own academic projects. I plan to use the improvements I build during GSoC in my future research projects, which gives me a personal stake in keeping the library robust, efficient, and scientifically accurate.

I enjoy the collaborative aspect of open source and plan to continue being an active presence in the SBI and Num Focus community.

Thank you for going through my proposal :)